

MEDSTAY

WHERE HOSTS MEET GUESTS

How We Built MEDSTAY

A complete walkthrough of the design, architecture, database, and deployment of a Libya-themed property-booking website — from an empty XAMPP folder to a live site at med-stay.xyz.

PHP 8

MySQL / MariaDB

Apache · XAMPP

Hostinger

Vanilla CSS

Technical build document · Generated June 2026 · Stack: LAMP (Windows-hosted dev)

01 What MEDSTAY Is

A booking platform for short-stay properties across Libya — guests browse curated listings, view a property in detail, and reserve dates instantly; hosts submit new properties and watch performance on a private dashboard.

The core idea

MEDSTAY is a server-rendered PHP website backed by a MySQL database. Every page that shows property data — the homepage cards, the listings grid, the detail page, the dashboard — reads live from the same three database tables. Nothing is hard-coded; editing a price means an **UPDATE** in the database, not a code change.

What a visitor can do

- **Browse** featured properties on the homepage and the full grid in `list.php`.
- **Inspect** a single property (photos, facts, description) on `property.php`.
- **Book** dates — with automatic protection against double-booking the same nights.
- **Submit** a property as a host, or send a message through the contact form.

What the owner can do

- Open `dashboard.php` to see total revenue, bookings, messages, average rating, recent reservations, and per-property performance — all computed with SQL aggregates.

Design language: Libyan cities (Tripoli, Benghazi, Misrata) and property types like *Courtyard House* and *Desert Camp*, dressed in a navy-and-gold palette with Reem Kufi headings and Tajawal body text — both fonts carry Latin and Arabic glyphs, leaving the door open for an Arabic / RTL version later.

The site map

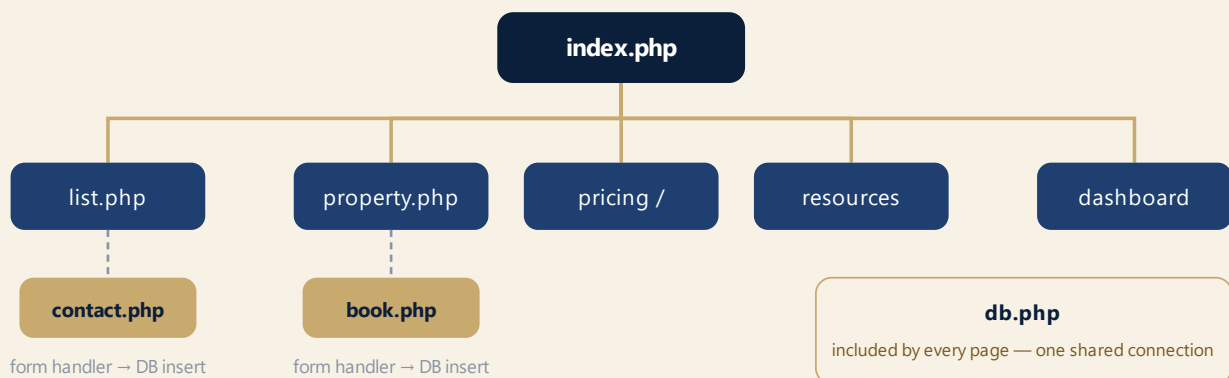


Figure 1 — Page map. Solid lines are navigation; dashed lines are form submissions to handlers.

02 The Technology Stack

A deliberately classic LAMP-style stack — chosen because it runs unchanged on cheap shared hosting and needs no build step.

Layer	Technology	Why it was chosen
Language	PHP 8.2	Server-side rendering, zero build tooling, runs everywhere.
Database	MySQL 8 / MariaDB 10.4	Relational data (properties ⇌ bookings) with foreign keys.
Web server	Apache (XAMPP) / LiteSpeed (live)	Bundled with XAMPP locally; LiteSpeed on Hostinger.
Front-end	Hand-written HTML + CSS	No framework — full control over the navy/gold look, tiny payload.
DB driver	mysqli (prepared statements)	Parameterised queries to block SQL injection.
Fonts	Reem Kufi + Tajawal (Google Fonts)	Latin + Arabic glyphs for a future RTL version.
Dev environment	XAMPP on Windows 11	Apache + MariaDB + phpMyAdmin in one install.
Production	Hostinger shared hosting	Files via File Manager, DB via phpMyAdmin.

No framework, on purpose

There is no React, no Laravel, no npm. Each page is a single `.php` file that queries the database at the top and prints HTML below it. The trade-off is more repetition, but the payoff is a site that deploys by copying files — no compilation, no `node_modules`, no surprises on a \$3/month host.

Where the code lives

File	Size	Role
<code>index.php</code>	~41 KB	Homepage: hero video, featured cards, contact form.
<code>list.php</code>	~30 KB	Full property grid + "list your property" host form.
<code>property.php</code>	~10 KB	Single-property detail page + booking form.
<code>dashboard.php</code>	~22 KB	Owner analytics dashboard (SQL aggregates).
<code>pricing.php</code> / <code>resources.php</code>	~27 KB ea.	Static marketing / info pages.
<code>book.php</code> / <code>contact.php</code>	small	POST-only form handlers (no UI of their own).
<code>db.php</code>	<1 KB	The shared, environment-aware database connection.

03 System Architecture

One request, three hops: the browser asks Apache, Apache runs the PHP file, the PHP file talks to MySQL, and finished HTML travels back. The same code runs in two environments without edits.

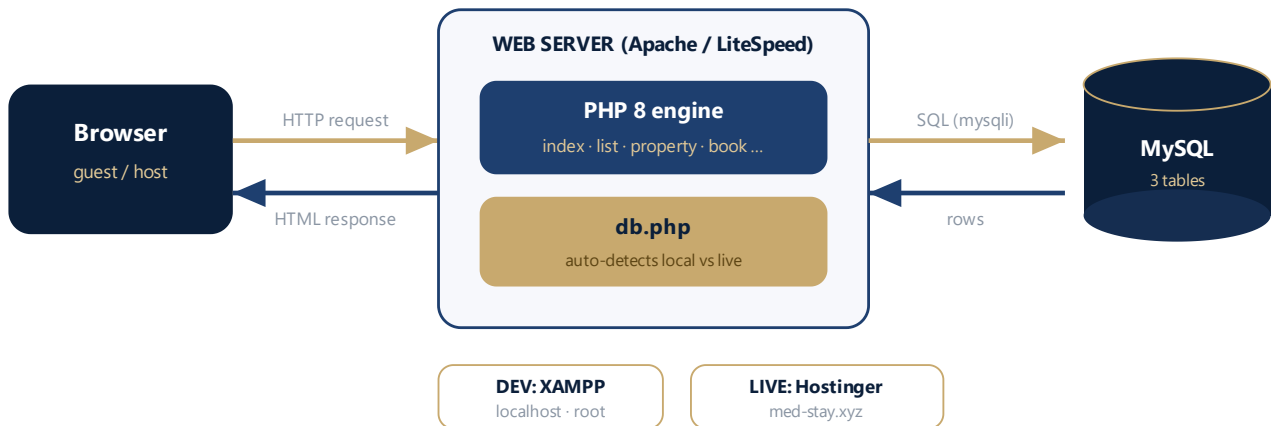


Figure 2 — Request lifecycle. The same files serve both the XAMPP dev box and the Hostinger live host.

One connection file, two environments

The trick that makes deployment painless is `db.php`. It inspects the incoming host name and picks the right credentials at runtime, so the *identical* file works on both machines — you never edit it before uploading.

```
// db.php - auto-selects LOCAL (XAMPP) vs LIVE (Hostinger)
$isLocal = php_sapi_name() === 'cli'
|| strpos($httpHost, 'localhost') !== false
|| strpos($httpHost, '127.0.0.1') !== false;

if ($isLocal) {
    // XAMPP
    $username = 'root'; $password = ''; $database = 'medstay';
} else {
    // Hostinger
    $username = 'u119737814_...'; $database = 'u119737814_medstay';
}
$conn = new mysqli($host, $username, $password, $database);
$conn->set_charset("utf8mb4"); // emoji + accents safe
```

Why this matters: with shared hosting there is no environment-variable panel and no `.env` tooling. Branching on `HTTP_HOST` is the simplest reliable way to keep one codebase for two databases.

04 Database Design

Three tables. `properties` is the catalogue; `bookings` records reservations and points back to a property; `contact_messages` stores form submissions.

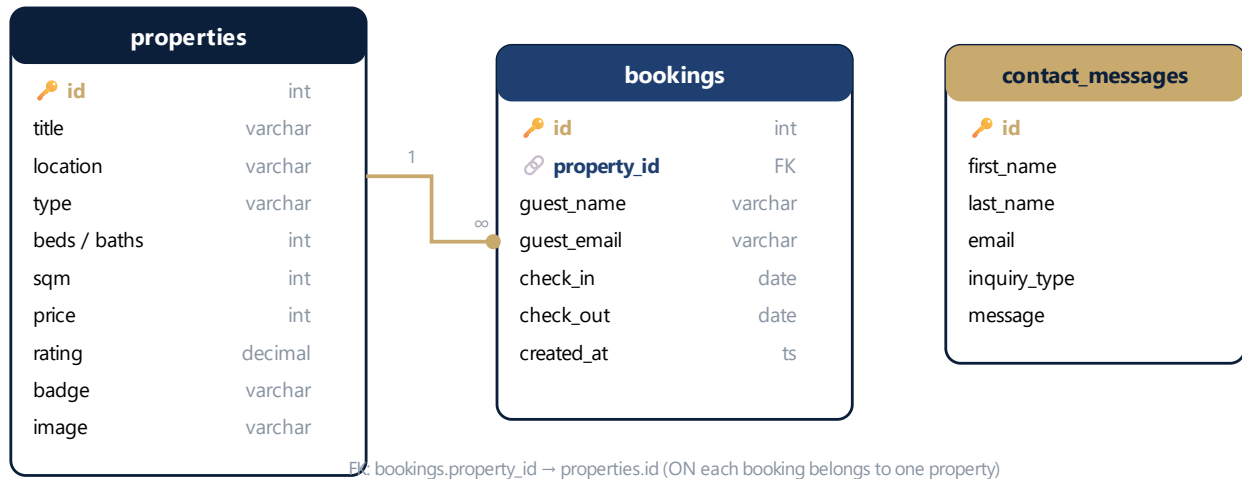


Figure 3 — Entity-relationship diagram. One property has many bookings; messages stand alone.

Why the relationship matters

The foreign key from `bookings.property_id` to `properties.id` is what lets the dashboard join the two tables and compute revenue per property: `nights × price` summed across every reservation. It also guarantees you can never book a property that doesn't exist.

```
-- the join that powers the dashboard's revenue figure
SELECT p.title,
       COUNT(b.id) AS bookings,
       SUM(DATEDIFF(b.check_out, b.check_in) * p.price) AS revenue
FROM properties p
LEFT JOIN bookings b ON b.property_id = p.id
GROUP BY p.id
ORDER BY revenue DESC;
```

Data lives in the DB, not the code. The nine seed properties (Tripoli apartments, Benghazi courtyard houses, Misrata desert camps) are rows in an `INSERT` statement. Changing a title or price on the live site means running an `UPDATE` in phpMyAdmin — editing the PHP alone does nothing.

05 The Booking Engine — Preventing Double-Bookings

The most interesting piece of logic in the project. Two guests must never reserve the same property for overlapping nights — even if they click "Reserve" at the same instant.

The naïve approach (and why it fails)

The obvious method is: *check* if dates are free, then *insert* the booking. But between the check and the insert, a second request can sneak in and book the same nights — a classic **race condition**. MEDSTAY avoids the gap entirely by making the check and the insert a **single atomic SQL statement**.

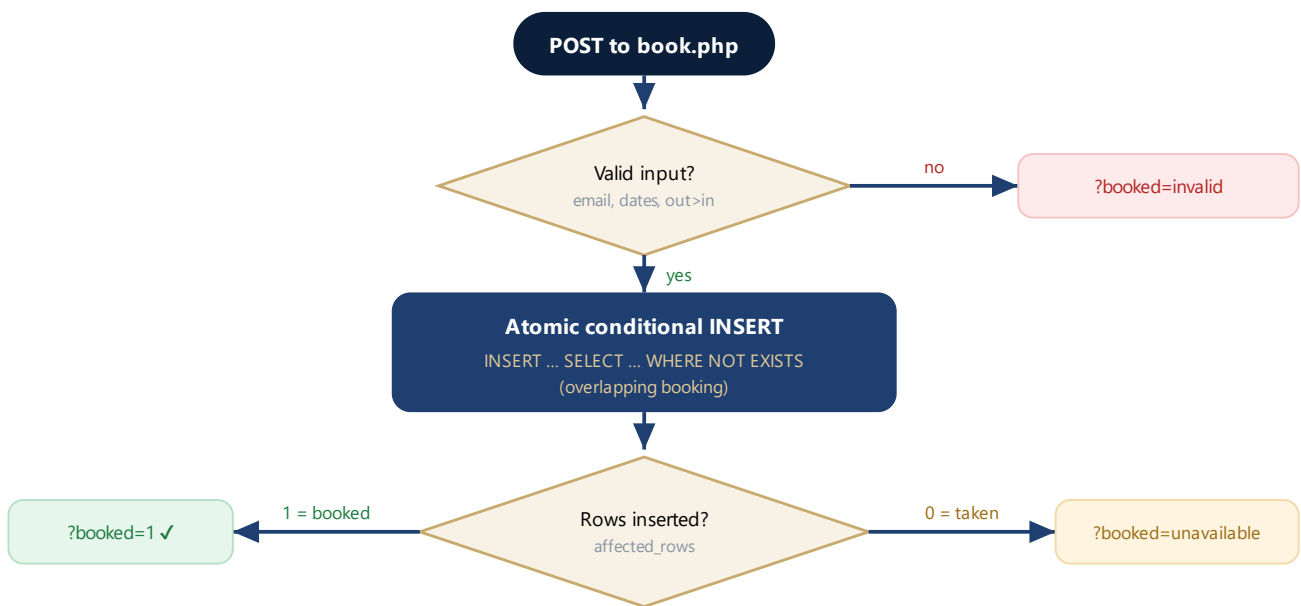


Figure 4 — Booking flow. The overlap check and the insert happen in one indivisible statement.

The atomic statement

The insert only writes a row when **no existing booking overlaps** the requested range. If a clash exists, zero rows are inserted and `affected_rows` tells us the dates were taken.

```
INSERT INTO bookings (property_id, guest_name, guest_email, check_in, check_out)
SELECT ?, ?, ?, ?, ?
FROM DUAL
WHERE NOT EXISTS (
    SELECT 1 FROM bookings
    WHERE property_id = ?
        AND check_in < ? -- existing starts before new ends
        AND check_out > ? -- existing ends after new starts
);
```

The overlap rule — `existing.check_in < new.check_out AND existing.check_out > new.check_in` — is the standard interval-overlap test. Note it deliberately allows *touching* dates: one guest's checkout day can be the next guest's check-in day.

Security built in: every value is bound through a prepared statement (`bind_param('isssiss', ...)`), so guest

names and emails can never be interpreted as SQL — injection is structurally impossible here.

06 The Build Process, Step by Step

How the project went from an empty folder to a working booking site.



Figure 5 — Build timeline.

1 · Establish the design language

Before any logic, the visual identity was fixed: a navy (`#0B1F3A`) and gold (`#C8A96E`) palette, Reem Kufi for headings, Tajawal for body. These live as CSS variables (`:root { --navy ... }`) repeated at the top of every page so the look stays consistent.

2 · Model the data in MySQL

Three tables were created in phpMyAdmin and seeded with nine real-feeling Libyan properties. The schema (and seed rows) is captured in `medstay.sql` so the whole database can be rebuilt from one file.

3 · Build the shared connection

`db.php` was written once with the local-vs-live auto-detection, then `include` d at the top of every page that touches data.

4 · Render the pages from data

Each page queries the database and loops over the results to print cards or detail blocks. All output goes through `htmlspecialchars()` to prevent broken markup and XSS.

5 · Wire up the form handlers

`book.php` and `contact.php` are POST-only endpoints. They validate, write to the database with prepared statements, then redirect back with a status flag in the URL (`?booked=1` , `?sent=error`) that the page reads to show a coloured alert — the **Post/Redirect/Get** pattern, which stops a refresh from re-submitting a booking.

6 · Deploy

Files are uploaded through Hostinger's File Manager; database changes are run in phpMyAdmin's SQL tab. Because `db.php` self-detects, nothing in the code changes between dev and live.

07 Deployment Workflow & Lessons Learned

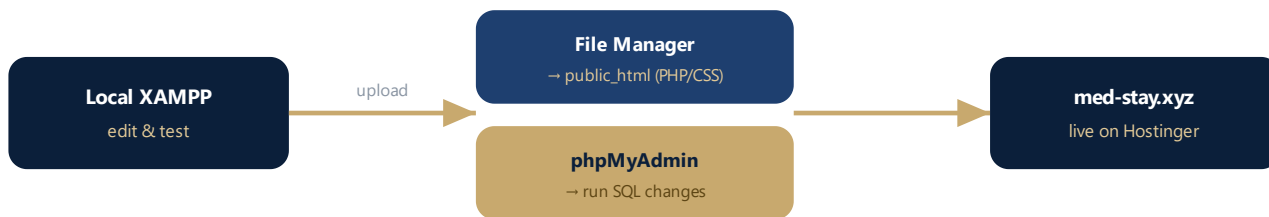


Figure 6 — Deploy path. Code and data travel through two separate channels.

The two-channel rule

Code and data are deployed **separately**. Uploading new PHP does *not* update the live database, and vice-versa. Because card content (cities, types, titles) comes from the database, a text change often means **both** a file upload and a phpMyAdmin **UPDATE**.

Gotchas that cost real time

Problem	Cause	Fix
Hero video 404 on live, fine locally	Linux is case-sensitive; path was <code>Videos/beach.mp4</code> with a capital V	Match filename casing exactly on the server
<code>localhost</code> connection fails	On the dev box <code>localhost</code> resolves to IPv6	Use <code>127.0.0.1</code> in <code>db.php</code>
Edits "don't show up" live	LiteSpeed page cache served a stale copy	<code>.htaccess</code> disables the cache for PHP pages
Text edit had no effect	That text is DB-driven, not in the PHP	Run the matching UPDATE on the live DB too

The cache-busting .htaccess

```
# Disable LiteSpeed cache + tell browsers not to cache PHP
<IfModule LiteSpeed>
  CacheDisable public /
</IfModule>
<FilesMatch "\.php$">
  Header set Cache-Control "no-store, no-cache, must-revalidate"
</FilesMatch>
```

What the stack does well

- **Zero build step** — deploy by copying files.
- **One codebase, two environments** via host-based detection.
- **Safe by construction** — prepared statements everywhere, output escaped.
- **Race-proof bookings** — overlap check and insert are atomic.

Where it would go next: host authentication for the dashboard (currently open), image uploads for host-submitted properties, an Arabic / RTL edition (the fonts already support it), and email confirmations on booking.

— End of document · MEDSTAY build walkthrough —